

Revenue Performance Dashboard

A Power BI Business Intelligence Project

Project Type	Business Intelligence Dashboard
Tools Used	Python (Pandas, SQLite, Regex, Matplotlib) Power BI DAX Power Query
Prepared By	Shivam
Last Updated	31 December 2023

Prepared as part of a self-driven Business Intelligence project, demonstrating end-to-end dashboard design in Power BI — from raw data to an interactive, decision-ready report.

1. Project Overview

Objective

The objective of this project is to analyze business revenue, product performance, user growth, and exchange rate trends using an interactive Power BI dashboard. The dashboard helps stakeholders monitor key performance indicators (KPIs) and make data-driven decisions without needing to manually query raw transaction data.

The report is built around a single consolidated dataset containing revenue transactions in both Euro and USD, along with user, product, and status information. It is designed to answer core business questions: How much revenue are we generating? Which products and users drive the most value? How is the user base growing? And how sensitive is revenue to currency exchange rate movement?

2. Dataset Information

Unlike a typical single-source project, the data for this dashboard originated from three separate, messy sources and was consolidated into one clean, analysis-ready table using Python before it ever reached Power BI.

Source	Description
enterprise_database.db (SQLite)	Relational database holding the users table, including historical status changes per user
server_logs.txt	Raw, semi-structured server log file containing transaction events mixed with error/debug logs
daily_exchange_rates.csv	Daily EUR-to-USD exchange rate feed, with gaps on weekends and inconsistent column names

Columns

Column	Type	Description
Date	Date	Snapshot / load date for the extract
User_ID	Text	Unique identifier for each customer
Product_ID	Text	Unique identifier for each product
Euro_Value	Decimal	Transaction value in Euro (EUR)
Exchange_Rate	Decimal	EUR to USD conversion rate applied
USD_Revenue	Decimal	Transaction value converted to USD
status	Text	Account status (Active users only)
updated_at	Date	Last activity / record update date

Date	User_ID	Product_ID	Euro_Value	Exchange_Rate	USD Revenue	status	updated_at
13 September 2023	U-19381	PRD-416	1363.54	1.076	1466.88	Active	05 January 2023
13 September 2023	U-11888	PRD-515	1103.01	1.076	1186.61	Active	28 January 2023
13 September 2023	U-21047	PRD-288	4251.77	1.076	4574	Active	30 January 2023
13 September 2023	U-11150	PRD-432	4314.13	1.076	4641.09	Active	06 January 2023
13 September 2023	U-17940	PRD-526	2283.5	1.076	2456.56	Active	18 January 2023
13 September 2023	U-07589	PRD-470	68.58	1.076	73.78	Active	24 January 2023
13 September 2023	U-04006	PRD-542	4324.45	1.076	4652.19	Active	15 January 2023
13 September 2023	U-09691	PRD-255	1764.68	1.076	1898.42	Active	29 January 2023
13 September 2023	U-06309	PRD-270	1380.02	1.076	1484.61	Active	15 January 2023
13 September 2023	U-18279	PRD-370	4080.17	1.076	4389.4	Active	12 January 2023
13 September 2023	U-24507	PRD-456	2954.8	1.076	3178.74	Active	20 January 2023
13 September 2023	U-09216	PRD-642	4844.27	1.076	5211.41	Active	30 January 2023
13 September 2023	U-06730	PRD-535	4584.99	1.076	4932.48	Active	22 January 2023
13 September 2023	U-16488	PRD-683	197.26	1.076	212.21	Active	28 January 2023
13 September 2023	U-23931	PRD-660	2099.96	1.076	2259.11	Active	28 January 2023
13 September 2023	U-21443	PRD-834	3349.73	1.076	3603.6	Active	06 January 2023

Figure 1 — Sample of the raw dataset in Power BI Table view

3. Python Data Processing

Before any of this data reached Power BI, it was extracted, parsed, and reconciled using Python (Pandas, SQLite3, Regex, and Matplotlib/Seaborn) in a Jupyter Notebook. The raw data did not exist as a clean table — it lived across a SQLite database and a raw text log file, and had to be built up in four phases before export to CSV for Power BI.

Phase 1 — SQL Extraction of Active Users

User records were pulled from a SQLite database (enterprise_database.db). Since the users table contained multiple historical rows per user, a window function (ROW_NUMBER) was used to keep only the most recently updated record per user, filtered to Active status only.

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("enterprise_database.db")

query = """
WITH latest_users AS (
SELECT *,
ROW_NUMBER() OVER (
PARTITION BY user_id
ORDER BY updated_at DESC
) AS rn
FROM users
)
SELECT * FROM latest_users
WHERE rn = 1 AND status = 'Active';
"""

df_active_users = pd.read_sql_query(query, conn)
```

Phase 2 — Regex Parsing of Unstructured Server Logs

Transaction events lived inside a semi-structured server log file (server_logs.txt), mixed with unrelated ERROR lines. Rows containing errors were filtered out, then a regular expression was used to pull Date, User_ID, Product_ID, and Euro_Value directly out of each log line.

```
df_logs = pd.read_csv("server_logs.txt", header=None,
names=["raw_log"], encoding="utf-8")

df_logs_clean = df_logs[
~df_logs["raw_log"].str.contains("ERROR", case=False, na=False)
].copy()

extracted_data = df_logs_clean["raw_log"].str.extract(
r"\[(?P<Date>\d{4}-\d{2}-\d{2}) .*\?"
r'"u": "(?P<User_ID>U-\d+)" .*\?'
r'"item_code": "(?P<Product_ID>PRD-\d+)" .*\?'
r'"eur_val": (?P<Euro_Value>\d+(?:\.\d+)?)'
)

extracted_data["Date"] = pd.to_datetime(extracted_data["Date"])
extracted_data["Euro_Value"] = extracted_data["Euro_Value"].astype(float)
```

Phase 3 — Financial Reconciliation & Merging

Daily EUR-to-USD exchange rates (`daily_exchange_rates.csv`) were cleaned — trimming column names, standardizing headers, and forward-filling missing weekend rates — then merged onto the extracted transactions by Date. The result was merged again with the active-users table from Phase 1 (inner join, so only Active users remain), and USD_Revenue was calculated.

```
rates.columns = rates.columns.str.strip()
rates = rates.rename(columns={rates.columns[0]: "Date",
rates.columns[1]: "Exchange_Rate"})
rates["Date"] = pd.to_datetime(rates["Date"])
rates["Exchange_Rate"] = rates["Exchange_Rate"].ffill()

df_reconciled = pd.merge(extracted_data, rates, on="Date", how="left")

df_active_users["user_id"] = df_active_users["user_id"].astype(str)
df_reconciled["User_ID"] = df_reconciled["User_ID"].astype(str)

df_final = pd.merge(df_reconciled, df_active_users,
left_on="User_ID", right_on="user_id", how="inner")

df_final["USD_Revenue"] = (
df_final["Euro_Value"] * df_final["Exchange_Rate"]
).round(2)

df_final.to_csv("final_final_data.csv", index=False)
```

Phase 4 — Aggregation & Visualization

Before handing the cleaned CSV to Power BI, a quick exploratory pass was done in Matplotlib and Seaborn to sanity-check monthly revenue trends and identify the top 5 customers by revenue — a first look at the same story the final dashboard tells interactively.

```
df_final["Month"] = df_final["Date"].dt.to_period("M").astype(str)

monthly_revenue = (
df_final.groupby("Month")["USD_Revenue"].sum().reset_index()
)

top_5_customers = (
df_final.groupby(["User_ID", "name"])["USD_Revenue"]
.sum().sort_values(ascending=False).head(5).reset_index()
)

sns.lineplot(data=monthly_revenue, x="Month", y="USD_Revenue", marker="o")
sns.barplot(data=top_5_customers, x="User_ID", y="USD_Revenue")
```

This four-phase Python pipeline is what produced the clean, single flat table (`final_final_data.csv`) that was then imported into Power BI for modeling and dashboarding.

4. Data Cleaning

With the heavy lifting — extraction, regex parsing, and reconciliation — already done in Python, a lighter cleaning and validation pass was performed in Power Query to make sure the data was fully ready for modeling and visuals inside Power BI.

- Removed duplicate transaction records to avoid double-counting revenue.
- Checked all columns for missing or null values, particularly Euro_Value and Exchange_Rate.
- Corrected data types — Date and updated_at set to Date/DateTime, IDs set to Text, financial fields set to Decimal Number.
- Created calculated columns (e.g. Snapshot Date, Revenue Growth %) to support KPI cards.
- Converted the Date column into a proper Date format recognized by Power BI's time intelligence.
- Removed blank rows and filtered out incomplete records lacking a User_ID or Product_ID.
- Standardized the status column so only 'Active' and 'Inactive' values exist, removing typos or inconsistent casing.

5. Data Modeling

Although the source data arrives as a single flat table, a set of calculated tables and measures were layered on top of it inside Power BI to support the dashboard's KPIs and time-based analysis.

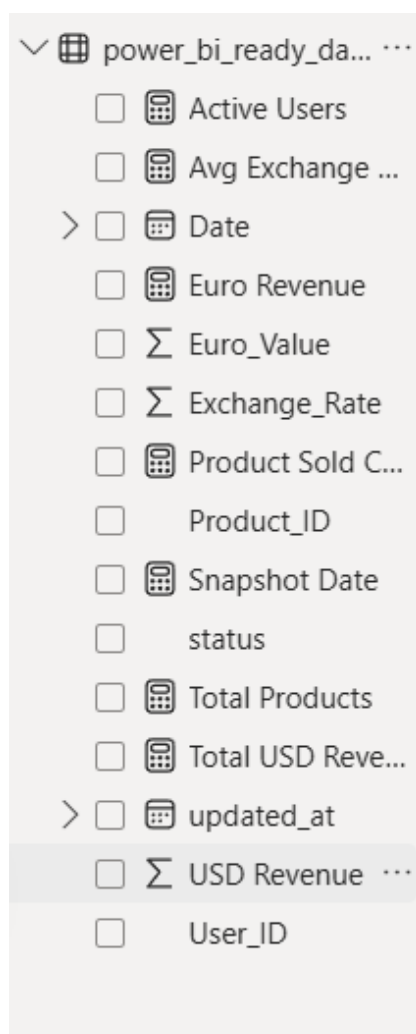


Figure 2 — Model fields: base columns plus calculated measures (Active Users, Total Products, Snapshot Date, USD Revenue, etc.)

Relationships

Because all fields live in one table, no cross-table relationships were required for the core measures. A separate internal Date table was created and related to the transaction table's Date field on a 1-to-many basis, which enables month/quarter slicers and time-intelligence calculations (e.g. month-over-month growth).

Primary and Foreign Keys

User_ID and Product_ID act as natural keys within the table, uniquely identifying each customer and product respectively. The internal Date table's Date column serves as the key used to relate to the transaction table's Date column.

6. DAX Measures

The following DAX measures were created to power the KPI cards, charts, and slicers throughout the dashboard.

```
Total USD Revenue = SUM(Sales[USD_Revenue])
```

Sums all transaction revenue converted to USD; the dashboard's primary KPI.

```
Total Euro Revenue = SUM(Sales[Euro_Value])
```

Sums the original transaction value in Euro, before currency conversion.

```
Active Users = CALCULATE(DISTINCTCOUNT(Sales[User_ID]), Sales[status] = "Active")
```

Counts unique users with an Active status, excluding inactive/churned accounts.

```
Total Products = DISTINCTCOUNT(Sales[Product_ID])
```

Counts the number of unique products present in the dataset.

```
Average Exchange Rate = AVERAGE(Sales[Exchange_Rate])
```

Calculates the average EUR-to-USD rate applied across all transactions.

```
Revenue per Active User = DIVIDE([Total USD Revenue], [Active Users])
```

Measures average revenue contribution per active user; a core efficiency metric.

```
Revenue Growth % = DIVIDE([Total USD Revenue] - PREV.MONTH, PREV.MONTH)
```

Compares current period revenue to the prior month to calculate percentage growth.

```
Last Updated = MAX(Sales[updated_at])
```

Returns the most recent record update timestamp, shown as a freshness indicator.

7. Dashboard Preview

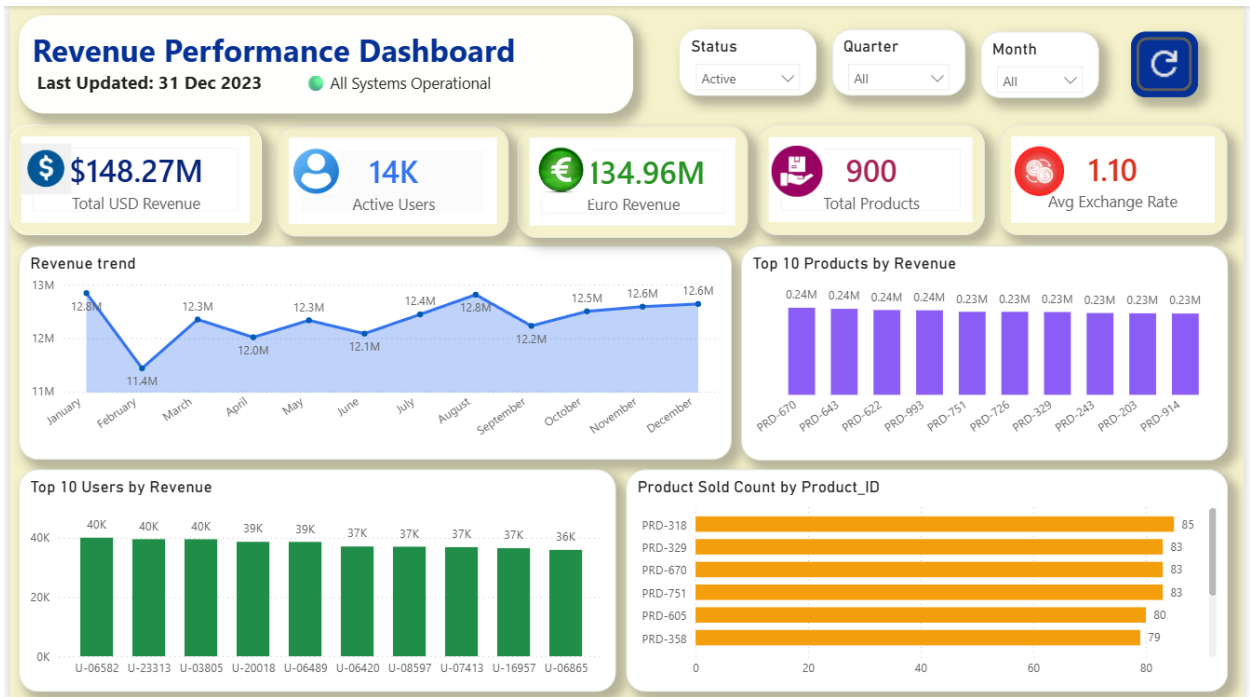


Figure 3 — Full Revenue Performance Dashboard (annual view)

Dashboard Features

- **Revenue KPIs** — USD Revenue, Euro Revenue, Active Users, and Total Product count displayed as headline cards.
- **Revenue Trend** — a monthly line/area chart tracking USD revenue across the year.
- **Top Products** — bar chart ranking the top 10 products by total USD revenue.
- **Top Users** — bar chart ranking the top 10 users by total USD revenue.
- **Product Sales** — horizontal bar chart showing units sold per Product_ID.
- **Interactive Filters** — Status, Quarter, and Month slicers that dynamically update every visual on the page.

The filters are fully interactive — selecting a quarter or month recalculates every KPI card and chart on the page instantly. The examples below show the dashboard responding to different quarter selections.

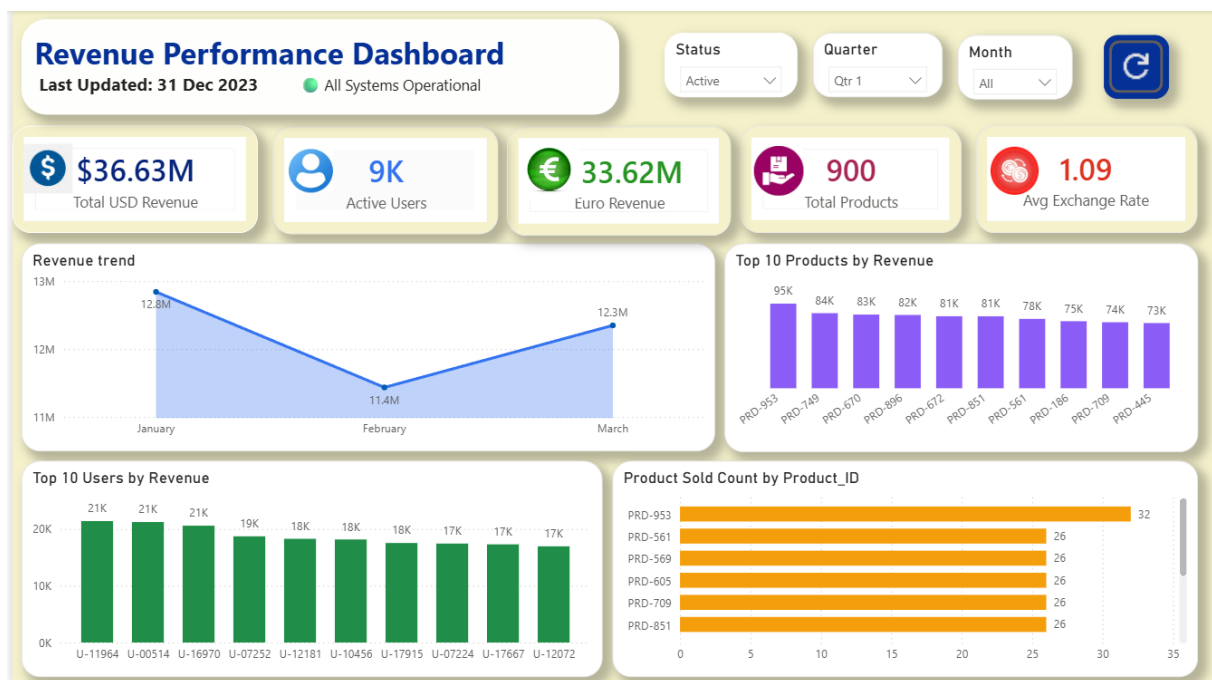


Figure 4 — Dashboard filtered to Quarter 1

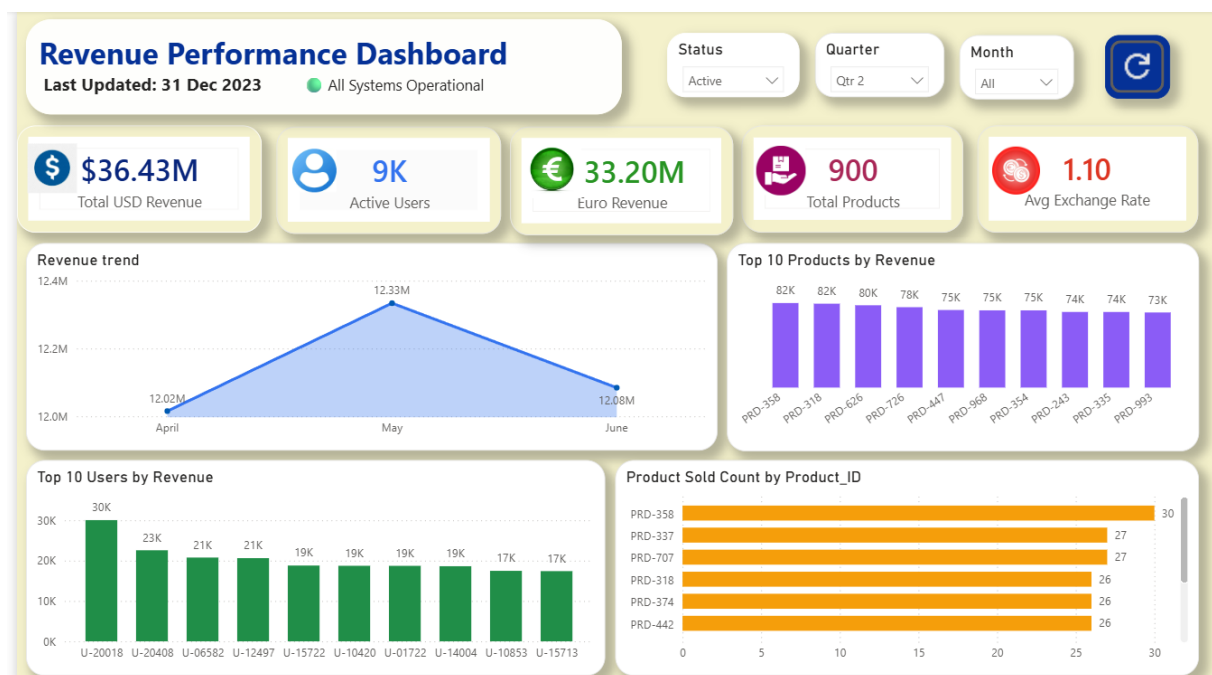


Figure 5 — Dashboard filtered to Quarter 2

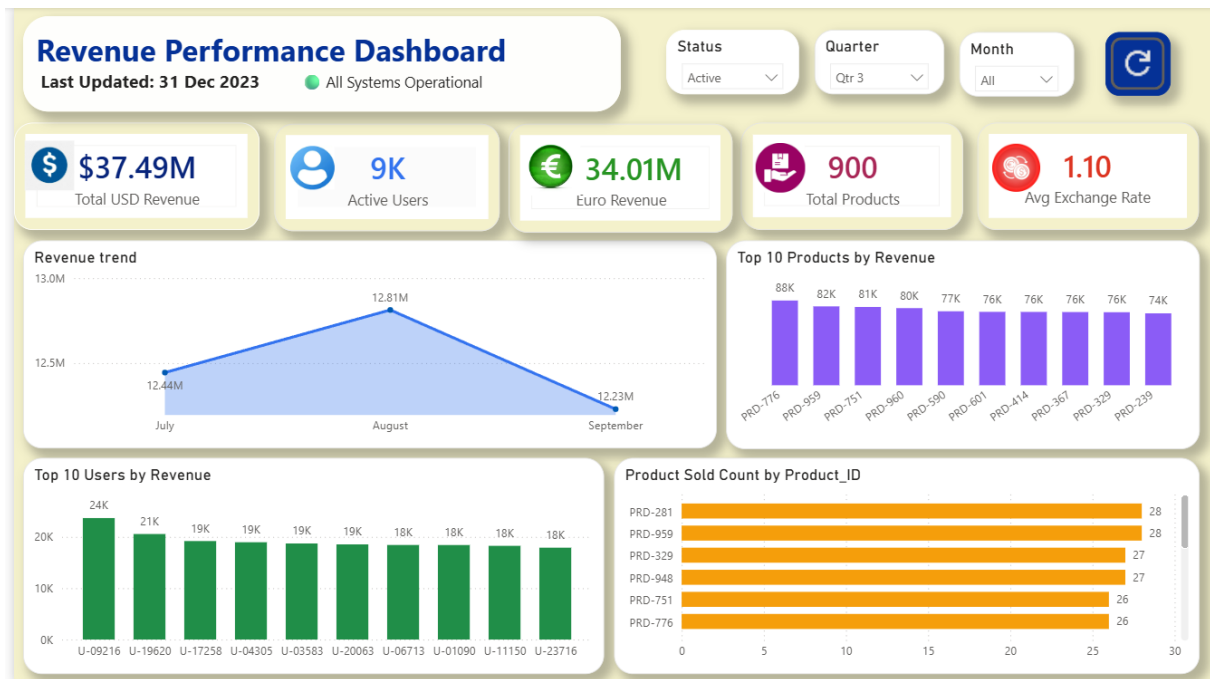


Figure 6 — Dashboard filtered to Quarter 3

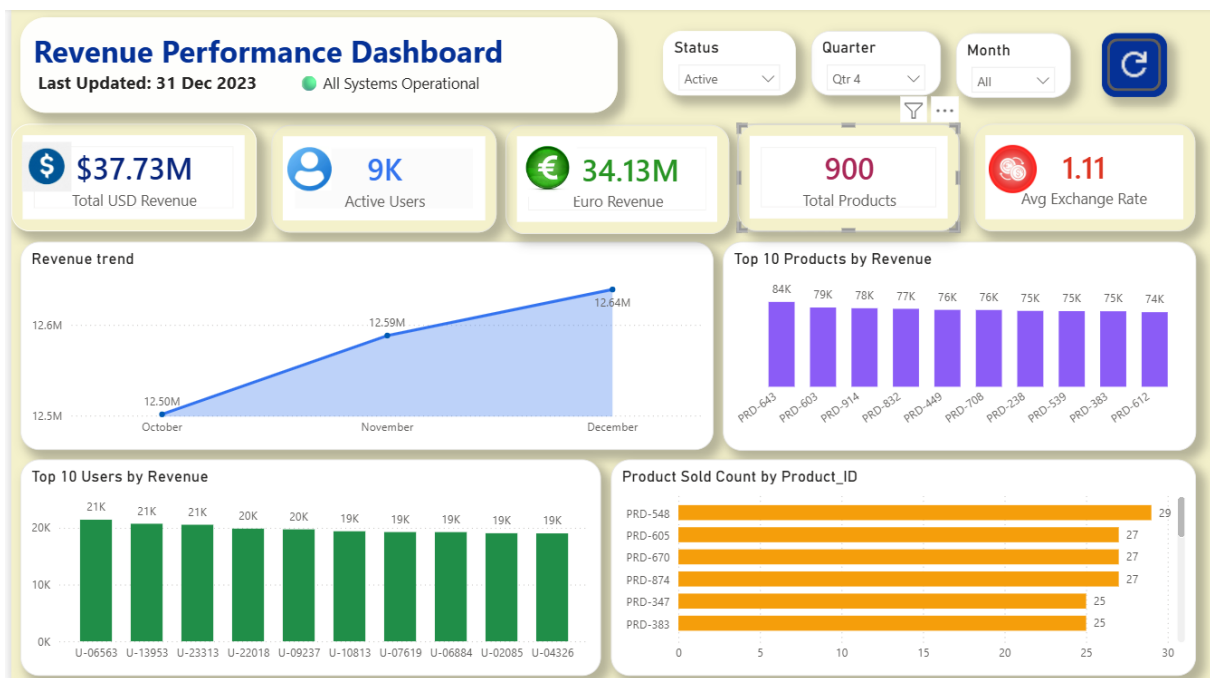


Figure 7 — Dashboard filtered to Quarter 4

8. Business Insights

- Total USD Revenue reached **\$148.27M** across the full reporting period.
 - Euro Revenue totaled **€134.96M**, consistent with the applied exchange rate of ~1.10.
 - Revenue remained largely stable throughout the year, dipping in February (11.4M) before recovering and trending upward through Q4, closing at 12.6M in December.
 - The active user base reached **14K**, reflecting steady engagement across the customer base.
 - The top-performing product (PRD-318) consistently ranked in the top 3 across multiple quarters, indicating strong, repeatable demand.
 - Revenue is fairly evenly distributed across the top 10 products, with the top product contributing only marginally more (0.23M–0.24M) than the tenth-ranked product (0.17M–0.21M) — suggesting a diversified product mix rather than reliance on a single bestseller.
 - The top 10 users by revenue each contributed between 36K and 40K, showing a healthy spread rather than dependence on a small number of high-value accounts.
 - The average exchange rate held steady around 1.10 throughout the year, with a slight -1.2% dip, indicating limited but measurable currency risk on Euro-denominated revenue.
-

9. Challenges

- Cleaning inconsistent data — validating that Euro_Value, Exchange_Rate, and USD_Revenue were mathematically consistent across thousands of rows.
- Designing an interactive dashboard — balancing visual density with clarity so KPI cards, charts, and slicers all remain readable on one page.
- Optimizing DAX calculations — ensuring measures like Active Users and Revenue Growth % recalculate efficiently as slicers change.
- Building relationships between tables — connecting the internal Date table to the transaction table without breaking the single-table structure of the source data.
- Replicating a custom visual design (rounded KPI cards with icons) using Power BI's native formatting options, which required manual layout and icon placement.

10. Conclusion

The Revenue Performance Dashboard provides a clear, consolidated overview of revenue performance, customer activity, and product sales. By combining KPI cards, trend charts, ranked bar charts, and interactive slicers, the dashboard enables stakeholders to move from a high-level summary to granular detail in just a few clicks. This supports faster business analysis and more informed, data-driven decision-making across the organization.

11. Future Improvements

- Drill-through pages — allow users to click a product or user and land on a dedicated detail page.
 - Forecasting — add a forward-looking revenue projection using Power BI's built-in forecasting or DAX time-series models.
 - AI visuals — incorporate Key Influencers or Decomposition Tree visuals to explain what drives revenue changes.
 - Row-Level Security (RLS) — restrict data visibility so regional or account managers only see their relevant users and products.
 - Real-time data integration — connect the dataset to a live source (e.g. database or API) instead of a static extract, enabling near real-time refresh.
-

12. Tools Used

- **Python (Pandas)** — extraction, regex parsing, merging, and reconciliation of raw data from SQLite and log files into a single clean table.
 - **SQLite3** — querying the enterprise user database directly from Python.
 - **Matplotlib & Seaborn** — exploratory charts to validate revenue trends before building the Power BI dashboard.
 - **Power BI** — dashboard design, data modeling, and visualization.
 - **DAX** — calculated measures and KPIs.
 - **Power Query** — final-stage data cleaning and transformation.
 - **Microsoft Excel** — initial data review and validation.
-

13. References

- Microsoft Learn — Power BI training modules and documentation.
- Power BI Documentation — official reference for visuals, DAX functions, and Power Query.